

第11章 动态内存管理

11.1 静态和动态内存

在计算机程序中，变量和对象存储在内存的不同区域，主要包括数据区、栈区和堆区。通常，我们将数据区和栈区的内存统称为静态内存，而将堆区的内存称为动态内存。“静态”意味着数据区或栈区中的数据在编译时就已经确定了相对于程序或当前栈顶位置的偏移量。相比之下，动态内存是在程序运行期间分配，其具体位置或在堆区中的偏移量在编译时无法预知。

以下代码示例展示了静态内存中变量或对象的创建和释放过程：

```

1  #include <iostream>
2  using namespace std;
3  int total = 100;
4  class A {
5  public:
6      A( int num = 0 ) :mNum( num ) { cout << "A::A(" << num << ")" << endl; }
7      ~A( ) { cout << "A::~~A( ),mNum=" << mNum << endl; }
8      int func( int n ) {
9          if ( n == 0 ) return 0;
10         int result = func( n - 1 ) + sBase;
11         static int count = 0;
12         ++count;
13         total += result;
14         cout << "count in func(int) = " << count << endl;
15         cout << "result of func(" << n << ") = " << result << endl;
16         cout << "current total = " << total << endl << endl;
17         return result;
18     }
19 private:
20     static int sBase;
21     int mNum;
22 };
23 A obj1( 1 );
24 const A obj2( 2 );
25 int A::sBase = 3;
26 int main( ) {
27     static int count = 8;
28     A myObj( 3 );
29     int val = myObj.func( 5 );
30     cout << "count in main = " << count << endl;
31     return 0;

```

```
32 }
```

运行结果:

```
A::A(1)
A::A(2)
A::A(3)
count in func(int) = 1
result of func(1) = 3
current total = 103
count in func(int) = 2
result of func(2) = 6
current total = 109
count in func(int) = 3
result of func(3) = 9
current total = 118
count in func(int) = 4
result of func(4) = 12
current total = 130
count in func(int) = 5
result of func(5) = 15
current total = 145
count in main = 8
A::~~A( ), mNum = 3
A::~~A( ), mNum = 2
A::~~A( ), mNum = 1
```

分析上述代码可知：全局变量 `total` 存储在数据区，全局对象 `obj1` 和全局常对象 `obj2` 也存储在数据区，类 `A` 中的静态成员变量 `sBase` 同样存储在数据区，此外，成员函数 `func` 中的静态局部变量 `count` 以及 `main` 函数中的静态局部变量 `count` 也都存储在数据区。数据区中的变量或对象的生命周期与程序运行周期一致，它们的内存会在程序结束时由系统自动回收。`main` 函数中的局部对象 `obj3` 和局部变量 `val` 存储在栈区，成员函数 `func` 中的局部变量 `result` 也存储在栈区，如果存在递归调用，`result` 变量会在栈区中多次分配，每次递归调用都会创建一个新的 `result` 实例，但其存储位置相对于当前调用栈的偏移是编译期可确定的，其生命周期与函数调用的生命周期一致，会在函数返回时自动释放。

使用静态内存具有一个重要的优势：无需手动管理变量或对象的释放。因为静态内存中的变量或对象，其释放操作总是由系统自动完成！但静态内存也存在一些局限性，这些不足正是引入动态内存的原因。下面列出静态内存的主要不足：

1. 静态内存的大小是有限制的。在数据区存储数据，会导致程序的大小变大，但实际上一个

可执行程序的大小是有限制的，特别是早期的编译器对数据区和代码区的大小都是有严格要求的。在栈区存储大数据，很容易造成栈溢出，虽然在编译时可指定栈的大小，但增加的栈空间也是很有限的。因此，静态内存不适合存放大对象。

2. 对象的生命周期不够灵活。对于数据区或栈区中的对象，它们的生命周期是固定的，程序员难以对其进行个性化控制。
3. 在静态内存中创建对象数组受到限制。首先，要求数组各维的大小必须都是编译期常量，这就难以创建大小可变的数组；其次，要求数组内的各项必须类型一致，大小一致，这限制了对象数组的使用，也是一般使用指针数组取代对象数组的原因；另外，对象数组要求各项必须可实例化，这意味着不允许定义抽象类数组，同时对于没有无参构造函数的类型也不太友好。例如：

```

1  class A {
2  public:
3      A( int n) {}
4      A( int n,int m ) {}
5  public:
6      static const int SIZE = 9;
7  };
8  const int SIZE = 7;
9  int main( ) {
10     int n = 10;
11     int arr00[n];          //非法, 静态数组的大小必为编译期常量
12     int arr01[n][5];       //非法, 静态数组的大小必为编译期常量
13     int arr02[ A::SIZE ] = { };      //合法
14     int arr03[ SIZE + A::SIZE ] { };  //合法
15     A arr04[3];             //非法, 缺少无参构造函数
16     A arr05[3] {};          //非法, 缺少无参构造函数
17     A arr06[3] = { A(1),A(2) };      //非法, 缺少无参构造函数
18     A arr07[3] = { A(1),A(2),A(3,4) }; //合法
19     A arr08[3] { A(1),A(2),A(3,4) }; //合法
20 }
```

4. 增加文件间编译期依赖性。使用静态内存可能需要明确的包含头文件，不能只使用类的前置声明。例如：

```

1  //SomeClass.h
2  #pragam once
3  #include "Object.h"
4  class SomeClass {
```

```

5  public:
6      void f( ) {
7          Object aObj;
8      }
9  private:
10     static Object sObj;
11     Object mObj;
12 };
13 //Object.h
14 #pragam once
15 class Object {
16 };
17 //AppMain.cpp
18 #include "SomeClass.h"
19 Object SomeClass::sObj;
20 int main( ) {
21     SomeClass instance;
22 }

```

5. 不适合处理动态数据结构。静态内存无法很好地支持动态数据结构(如链表、树、图等),因为这些结构的大小和形态在编译时无法确定,需要在运行时动态调整。

要克服静态内存的不足,可以使用动态内存,即在运行时刻,按照程序员的意图在堆区中创建和释放对象。相对于静态内存,使用动态内存可以提供更大的灵活性,包括对象的创建时间和释放时间,创建的对象个数,对象的实例化参数等。

11.2 单对象的动态创建和释放

11.2.1 动态分配

要在堆区中动态创建单一的变量或对象,可使用 `new` 表达式,其语法格式为:

格式 1: `[::] new T [new 初始化]`

格式 2: `[::] new (T) [new 初始化]`

其中, `new` 初始化部分是可选的。如果没有提供,则进行缺省初始化;如果提供了初始化,又分两种形式:一种是由圆括号包围初始值列表,称为直接初始化;另一种是由花括号包围初始值列表,称为列表初始化。例如:

```

1  int * p1 = new int( 5 );           //值初始化
2  const int * const p2 = new int( *p1 ); //值初始化
3  A * p3 = new A;                   //值初始化+缺省初始化

```

```

4  const A * p4 = new A( 100, 200 );      //值初始化+直接初始化
5  const A * p5 = new A { 100,200 };      //值初始化+直接列表初始化
6  const A * *p6 = new A * ( p4 );       //值初始化

```

先看普通 new 表达式的执行过程。以表达式 new T(3)或::new T(3)为例，其执行过程大致分三步：

- (1) 若表达式为 new T(3)，则调用类 T 中的类方法 static void * operator new(size_t size)，尝试分配空间；若表达式为::new T(3)，则直接调用全局的::operator new(size_t size)。若分配失败，转到异常处理。成功则继续。
- (2) 执行类 T 的相应构造函数。若构造失败，查找适当函数释放上一步中已分配的空间，并抛出异常。否则，继续。
- (3) 将 void *指针转换成 T *指针，并返回。

上述过程的伪代码为：

```

1  void * ptr = T::operator new( sizeof( T ) );
   或者 void * ptr = ::operator new( sizeof( T ) );
2  执行类 T 的构造函数，相当于调用 T::T( ptr , 3 );，这里的 ptr 就是构造函数的 this 指针
3  将 ptr 强制转换为 T* 类型并返回，相当于 return (T*) ptr;

```

下面针对上述过程给出几点说明：

1. 处理 new 表达式的核心过程是调用 operator new 的过程。在 C++中有两类 operator new，一类是全局名字空间下的 operator new，一类是类级的 operator new。全局名字空间下的 operator new 实现通常由编译器提供，对于类级的 operator new，如果用户没有定义，编译器也会提供缺省的实现。
2. 类级的 operator new 成员函数，一定是类方法。即使没有显式给出 static 关键字，也是类方法。
3. 对于表达式 new T(3)，T 是类类型，那么会执行 T::operator new 函数；对于 new int(3)，执行的是 ::operator new。
4. 对于表达式 ::new T(3)，编译器会忽略类级的 operator new，直接使用全局的，即::operator new。
5. 类级的 operator new，可以通过重载 operator new 运算符进行自定义。
6. 当 operator new 失败，如内存不足时，用户可自定义处理函数进行必要的处理，如回收内

存, 转存内存数据等。这种处理函数可以使用系统的库函数 `set_new_handler` 设置, 也可直接使用异常处理(详见第 21 章异常的处理)。例如:

```

1  #include<iostream>
2  #include<cstdlib>    // std::exit
3  #include<new>        // std::set_new_handler
4  void no_memory() {
5      //自定义的错误处理: 如提示、回收内存、数据转存等
6      std::cout << "分配内存失败!\n";
7      std::exit(1);
8  }
9  int main() {
10     using Handler = void (* )();
11     Handler hOld = std::set_new_handler( no_memory );
12     std::cout << "尝试分配 1G 内存...";
13     char * p =new char[1024 * 1024 * 1024];
14     std::cout << "分配成功!\n";
15     delete[] p;
16     set_new_handler(hOld);    //恢复为原处理函数
17 }
```

11.2.2 示例

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Person;
5  class Person {
6  public:
7      Person( const string & name ) :mName( name ) { }
8      const string & getName( ) const { return mName; }
9  private:
10     string mName;
11 };
12 class Dog {
13 public:
14     Dog( const string & name ) {
15         cout << "构造 Dog 且动态分配 Person" << endl;
16         mpOwner = new Person( name );
17     }
18     ~Dog( ) {
19         cout << "析构 Dog 且动态释放 Person" << endl;
20         delete mpOwner;
21     }
```

```

22     Person * owner( ) const { return mpOwner; }
23 private:
24     Person * mpOwner;
25 };
26 int main( ) {
27     Dog * pDog = new Dog( "张三" );
28     Person * pOwner = pDog->owner( );
29     cout << pOwner->getName( ) << endl;
30     delete pDog;
31 }

```

栈区中只创建了两个指针变量，分别指向一只狗和主人，而狗和主人都创建在堆区。狗和主人的生命周期完全由程序决定：进入 `main` 函数时，创建狗对象；创建狗对象时，创建主人对象；当 `main` 函数结束前，释放狗对象；释放狗对象之前，先释放主人对象。`main` 函数结束时，进行清栈操作，清除栈区中两个指针变量。

11.2.3 动态释放

由于堆区内存是全体程序共享的，应用程序不会自动释放，因此需要显式地释放由 `new` 表达式分配的堆区内存。为此，可以使用 `delete` 表达式来释放动态分配的内存，语法格式为：

格式： `[::] delete 表达式`

对于 `delete ptr`；其执行过程如下：

- (1) 若 `ptr` 为 `nullptr`，则结束
- (2) 执行析构函数 `T::~T()`。
- (3) 回收动态内存。若存在匹配的类级 `operator delete` 函数，则执行；若不存在，则执行全局名字空间的 `::operator delete` 函数。

其伪代码为：

- (1) `if (nullptr == ptr) return;`
- (2) 执行 `T::~T(ptr)`
- (3) 执行 `T::operator delete(ptr)` 或 `::operator delete(ptr)`

11.3 数组的动态创建和释放

在堆区还可以动态地创建和释放数组。语法格式为：

动态创建：

格式 1: `[::] new T [new 初始化]`

格式 2: `[::] new (T) [new 初始化]`

动态释放:

格式 1: `[::] delete[] expression`

动态创建数组与创建单个对象的主要区别在于: T 必须是数组类型, 数组的元素可以是内置类型、指针、自定义类型, 甚至是函数指针或另一个大小固定的数组。在释放内存时, 必须使用 `delete[]` 形式, 以区别于单个对象的释放, 确保正确且完整地释放堆空间。例如:

```

1  #include <string>
2  using namespace std;
3  class Person {
4  public:
5      Person( int age = 18 , const string & name="某某") :mAge( age ),mName(name) { }
6  private:
7      int      mAge;
8      string   mName;
9  };
10 int main( ) {
11     int n = 3;
12     int * p1 = new int[ n ];
13     delete[ ] p1;
14     int * p2 = new int[ 3 ] { 1,2,3 };
15     delete[ ] p2;
16     int ** p3 = new int * [ 5 ];
17     delete[ ] p3;
18     Person * p3 = new Person[ n ];      //缺省初始化, 需无参构造函数
19     delete[ ] p3;
20     //列表初始化, 第二个参数使用缺省值
21     Person * p4 = new Person[ 3 ] { 18,19,20 };
22     delete[ ] p4;
23     //列表初始化
24     Person * p5 = new Person[ 4 ] { {19,"张三"},{20,"李四"},{19,"王五"},{21,"赵六"} };
25     delete[ ] p5;
26     Person ** p6 = new Person * [ n ];
27     delete[ ] p6;
28     //以下三种写法是等价的, 都是动态创建一个指针数组, 其中每个元素是一个函数指针
29     typedef int ( *Func1 )( );          //旧风格, 定义函数指针类型
30     Func1 * p7 = new Func1[ n ];
31     delete[ ] p7;
32     using Func2 = int ( * )( );         //Func2 的类型同 Func1
33     Func2 * p8 = new Func2[ n ];

```



```

34     delete[ ] p8;
35     Func1 * p9 = new ( int ( *[ n ] )( ) ); //等价于 p7 或 p8 的定义
36     delete[ ] p9;
37 }

```

11.3.1 数组的动态创建和释放

数组动态创建的常用语法形式是 `new T[size]`，相比于创建静态数组，动态创建的数组大小可以是变量。如果 `T` 本身就是一个数组，那么 `T` 的元素个数必须是编译期常量。例如：

```

1  int main( ) {
2      const int N = 4; //N 是编译期常量
3      int m = 4;      //m 是变量
4
5      int arr1[10];      //合法
6      int arr2[N] {1,2,3,4}; //合法
7      int arr3[N] = {1,2,3,4}; //合法
8
9      int arr04[N][3] = {{1,2,3},{4,5,6},{7,8,9},{10,11,12}}; //合法
10     int arr4[m];      //非法, m 不是编译期常量
11     //注意 new int[m][3]的形式, int[m][3]的编译器解释是 (int[3])[m]
12     auto arr5 = new int[m][3] {{1,2,3},{4,5,6},{7,8,9},{10,11,12}}; //合法
13     delete[] arr5;
14
15     int * arr6 = new int[m]; //合法
16     delete[] arr6;
17 }

```

当使用 `new T[n]` 分配数组时，其语义过程类似单对象的动态分配过程，也分为几步：

- (1) 调用 `void * T::operator new[ ] (size_t size );` 函数，尝试分配空间，若失败则转到异常处理；成功则继续。这里 `size` 值为 `sizeof( T ) * n + overhead`，其中，`overhead` 表示为记录数组大小而额外增加的字节数。
- (2) 根据数组元素的个数，执行多次类 `T` 的构造函数。若任意一个构造失败，则查找适当函数释放上一步中分配的空间。
- (3) 将 `void *` 指针转换成 `T *` 指针，并返回。

同样地，使用 `delete[ ] pointer;` 释放动态数组的过程，也分几步：

- (1) 若 `pointer` 为 `nullptr`，直接返回。
- (2) 执行多次的析构函数。

(3) 执行 `operator delete[ ] (void * pointer)`或`::operator delete[ ] (void * pointer)` 回收内存。

11.3.2 示例

定义含有多张 Card 的 Poke 类。Card 类函数，如下：

```
1 class Card {
2 public:
3     Card(int id) :mID(id) { }
4     //其它略
5 private:
6     const int mID;
7 };
```

那么下面的代码是不能编译通过的：

```
1 class Poke {
2 public: /* 略 */
3 private:
4     Card mCards[54];
5 };
6 int main( ) {
7     Poke aPoke;           //非法，缺少 Card 类无参构造函数
8     Poke * pPoke = new Poke; //非法，缺少 Card 类无参构造函数
9     delete pPoke;
10 }
```

无论是在栈区还是堆区创建 Poke 类对象，都要求 Card 类具有无参构造函数。然而，为 Card 类提供无参构造函数在实际应用中并不合理，因为一副扑克中的每张牌应该是独一无二的，它们的 mID 值应当各不相同。为此，可以使用指针数组重新定义 Poke 类：

```
1 class Poke {
2 public:
3     Poke( ) {
4         for ( int i = 0; i < SIZE; ++i ) mpCards[ i ] = new Card( i + 1 );
5     }
6     ~Poke( ) {
7         for ( int i = 0; i < SIZE; ++i ) delete mpCards[ i ];
8     }
9 private:
10     static const int SIZE = 54;
11     Card * mpCards[ SIZE ];
12 };
13 int main( ) {
```

```

14     Poke aPoke;           //合法
15     Poke * pPoke = new Poke; //合法
16     delete pPoke;
17 }

```

若需要使用多副扑克组成的 Poke，则可如下定义：

```

1  class Card {
2  public:
3      Card( int id ) :mID( id ) { }
4      //其它略
5  private:
6      const int mID;
7  };
8  class Poke {
9  public:
10     Poke( int pokeNum ) : mCardCount( pokeNum * SIZE ) {
11         mpCards = new Card * [ mCardCount ];
12         for ( int i = 0; i < mCardCount; ++i )
13             mpCards[ i ] = new Card( i % SIZE + 1 );
14     }
15     ~Poke( ) {
16         for ( int i = 0; i < mCardCount; ++i )
17             delete mpCards[ i ];
18         delete[ ] mpCards;
19     }
20 private:
21     static const int SIZE = 54;
22     const int mCardCount;
23     Card **  mpCards;
24 };
25 int main( ) {
26     Poke aPoke( 2 );           //合法
27     Poke * pPoke = new Poke( 2 ); //合法
28     delete pPoke;
29 }

```

11.4 三点说明

1. 在使用 new 表达式和 delete 表达式时，应相互匹配。使用 new 创建的，用 delete 释放；使用 new[] 创建，则用 delete[] 释放。若 new 和 delete 不匹配，可能会造成释放不完整以及内存泄漏。

2. 使用 `new` 或 `new [ ]` 进行动态创建时，会调用构造函数。若构造失败，会释放对象本身已分配的动态内存，但不保证释放构造过程分配的其它堆内存。相对应地，使用 `delete` 或 `delete[ ]` 释放时，会调用析构函数，若析构函数失败会怎样呢？结论是会造成内存泄漏。因此，在定义类的构造函数和析构函数时，应保证：构造函数可以产生异常，但析构函数总是无异常产生的，即析构函数的实现应等价于 `T::~T() noexcept` 的函数实现。
3. 注意到分配动态数组时，采用的语法格式是 `new T[size];`，语法中由 `size` 指明了数组的大小，但当释放时，使用的格式是 `delete[ ] pointer;`，这里没有指明数组的大小，那怎么确定析构的次数以及回收的内存大小呢？实际上，当执行 `new T[size]` 时，真实分配的内存大小是 `sizeof(T) * size + overhead`，`overhead` 作为额外的内存开销可用于记录数组大小等信息，所以当使用 `delete[ ] pointer;` 释放数组时，可由 `pointer` 自动查询到这些额外信息。

11.5 重载 `operator new / operator delete / operator new[ ] / operator delete[ ]`

通过前面的说明，可以看到在使用 `new` 和 `delete` 表达式进行动态创建和释放时，其实际的执行过程会用到 `operator new / operator delete / operator new[ ] / operator delete[ ]`。这些函数分成两类，一类是全局名字空间的，一类是类级的。当使用 `new T` 或 `new T[ n ]` 表达式分配动态内存时，会调用类级的 `T::operator new` 或 `T::operator new[ ]`；当使用 `::new T` 或 `::new T[ n ]` 时，会调用全局名字空间的 `::operator new` 或 `::operator new[ ]`；同理，当使用 `delete pointer` 或 `delete[ ] pointer` 释放动态内存时，会调用类级的 `T::operator delete` 或 `T::operator delete[ ]`；当使用 `::delete pointer` 或 `::delete[ ] pointer` 时，会调用全局名字空间的 `::operator delete` 或 `::operator delete[ ]`；

对于一个类型 `T`，如果没有显式定义 `operator new / operator delete / operator new[ ] / operator delete[ ]`，编译器会提供缺省的。一般情况下，不必显式地重载 `operator new / operator delete / operator new[ ] / operator delete[ ]`，使用编译器提供的缺省即可。只有在特殊目的时，才有重载的必要。若重载，其各种重载形式的语法格式为：

格式 1:	<code>void * operator new (std::size_t size)</code>
格式 2:	<code>void operator delete (void * ptr) noexcept</code>
格式 3:	<code>void * operator new[] (std::size_t size)</code>
格式 4:	<code>void operator delete[] (void * ptr) noexcept</code>

下面是这几类重载函数的说明：

- 重载的 `operator new / operator delete / operator new[ ] / operator delete[ ]` 必须是类方法，即使省

略 `static` 关键字，也是类方法。

- 格式(1)和格式(2)用于单个 `T` 类对象的动态创建和释放；格式(3)和格式(4)用于 `T` 类型数组的动态创建和释放。
- 重载时，要保证同时重载 `operator new` / `operator delete` / `operator new[ ]` / `operator delete[ ]` 四类函数。例如，若只重载了 `operator new`，没有重载 `operator delete`，那么当释放时，在类 `T` 中查找不到匹配的 `operator delete`，会使用全局的 `::operator delete`，这就可能出现语义不匹配的情况；对于 `operator new[ ]` 和 `operator delete[ ]` 也是同样道理。此外，若只重载 `operator new` / `operator delete`，而没有重载 `operator new[ ]` / `operator delete[ ]`，也会造成单对象和对象数组在分配和释放上语义的不匹配。
- 重载的各 `operator delete` 和 `operator delete[ ]` 函数，应带有 `noexcept` 异常说明，即要求 `operator delete` 函数必须是无异常的，也就是要保证回收内存总是成功的。

下面的示例代码，演示了自定义重载的 `operator new` 和 `operator delete`，以及如何使用。

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Person {
5  public:
6      Person( int age = 0 ) :mAge( age ) { }
7      static void * operator new( size_t size ) {
8          return new char[ size + 16 ]; //或 return ::new char[ size + 16 ];
9      }
10     static void operator delete( void * ptr ) noexcept {
11         delete[ ] ptr; //或::delete[ ] ptr;
12     }
13     static void * operator new[ ]( size_t size) {
14         return new char[ size ];
15     }
16     static void operator delete[ ]( void * ptr) noexcept {
17         delete[ ] ptr;
18     }
19     static void * operator new( size_t size, int added ) {
20         char * p = new char[ size + sizeof(int) ];
21         *( int * ) ( p + size ) = added;
22         return p;
23     }
24     static void operator delete( void * ptr, int added ) noexcept {
25         delete[ ] ptr;

```

```

26     }
27     static void * operator new( size_t size, const string & nation ) {
28         cout << "Nation = " << nation << endl;
29         return new char[ size ];
30     }
31     static void operator delete( void * ptr, const string & nation ) noexcept {
32         delete[ ] ptr;
33     }
34 private:
35     int mAge;
36 };
37 int main( ) {
38     Person * p1 = new Person( 18 );
39     delete p1;
40     Person * p2 = new( 10 ) Person( 18 );
41     int * pp = ( int * ) p2;
42     cout << *pp << endl;           //输出 18
43     cout << *( pp + 1 ) << endl;   //输出 10
44     delete p2;
45     Person * p3 = new( "中国" ) Person( 18 ); //输出 Nation = 中国
46     delete p3;
47 }

```

这里要注意, 由于用户参数列表可不同, 重载的函数可以同时存在多个, 但在定义 `operator new` 和 `operator delete` 时, 应成对地重载对应参数列表的 `operator new` 和 `operator delete`, 例如上面示例代码中的多个 `operator delete` 函数, 虽然实现一样, 也应分别提供。

11.5.1 示例

在程序的调试和调优过程中, 常需要确定对象的内存占用情况, 包括对象的实时个数, 累计个数, 实时存储空间, 累计存储空间等, 这时使用重载的 `operator new` /`operator delete` /`operator new[ ]` /`operator delete[ ]` 就是一种较好的解决方案。

```

1  #include <iostream>
2  using namespace std;
3  #define DEBUG //测试期间使用。测试完毕后, 可注释掉
4  class A {
5  public:
6  #ifdef DEBUG
7      static void * operator new ( size_t size ) {
8          //多分配一个 size_t, 用于存储字分配的字节数
9          char * pHeap = new char[ size + sizeof( size_t ) ];

```

```

10     *( size_t * ) ( pHeap ) = size;    //记录分配的字节数
11     ++sTotalNewCount;                  //累计个数+1
12     ++sActiveCount;                    //当前存活对象个数+1
13     sActiveMemory += size;              //当前存活对象所占内存数, 增加 size 字节
14     if ( sMaxMemory < sActiveMemory )
15         sMaxMemory = sActiveMemory;    //更改内存占用的最大值
16     return pHeap + sizeof( size_t );
17 }
18 static void operator delete( void * ptr ) noexcept {
19     char * pHeap = ( ( char * ) ptr - sizeof( size_t ) );
20     size_t size = *( ( size_t * ) pHeap );
21     --sActiveCount;                     //当前存活对象个数-1
22     sActiveMemory -= size;              //当前存活对象所占内存数
23     ++sTotalDeleteCount;                //累计释放的对象个数+1
24     delete[ ] pHeap;
25 }
26 static void * operator new[ ]( size_t size ) {
27     //多分配一个 size_t,用于存储字分配的字节数
28     char * pHeap = new char[ size + sizeof( size_t ) ];
29     *( size_t * ) ( pHeap ) = size;    //记录分配的字节数
30     sTotalNewCount += size / sizeof( A ); //累计个数+数组大小
31     sActiveCount += size / sizeof( A );   //当前存活对象个数+数组大小
32     sActiveMemory += size;                //当前存活对象所占内存数, 增加 size 字节
33     if ( sMaxMemory < sActiveMemory )
34         sMaxMemory = sActiveMemory;    //更改内存占用的最大值
35     return pHeap + sizeof( size_t );
36 }
37 static void operator delete[ ]( void * ptr ) noexcept {
38     char * pHeap = ( ( char * ) ptr - sizeof( size_t ) );
39     size_t size = *( ( size_t * ) pHeap );
40     sActiveCount -= size / sizeof( A );   //当前存活对象个数-数组大小
41     sActiveMemory -= size;                //当前存活对象所占内存数
42     sTotalDeleteCount += size / sizeof( A ); //累计释放的对象个数+数组大小
43     delete[ ] pHeap;
44 }
45 #endif
46 public:
47     static void showInfo( ) {
48 #ifdef DEBUG
49         cout << "累计创建的对象个数 = " << sTotalNewCount << endl;
50         cout << "累计释放的对象个数 = " << sTotalDeleteCount << endl;
51         cout << "当前存活的对象个数 = " << sActiveCount << endl;
52         cout << "存活对象所占内存数 = " << sActiveMemory << endl;
53         cout << "所占用内存的最大值 = " << sMaxMemory << endl;

```

```

54     cout << endl;
55 #endif
56 }
57 private:
58     int mVal = 100;
59 public:
60 #ifdef DEBUG
61     static size_t sActiveCount;        //当前存活对象个数
62     static size_t sActiveMemory;       //当前存活对象所占内存数
63     static size_t sTotalNewCount;      //累计创建的对象个数
64     static size_t sTotalDeleteCount;   //累计释放的对象个数
65     static size_t sMaxMemory;          //运行期间，占用内存的最大值
66 #endif
67 };
68 #ifdef DEBUG
69 size_t A::sActiveCount = 0;
70 size_t A::sActiveMemory = 0;
71 size_t A::sTotalNewCount = 0;
72 size_t A::sTotalDeleteCount = 0;
73 size_t A::sMaxMemory = 0;
74 #endif
75 int main( ) {
76     A * p1 = new A;
77     A::showInfo( );
78     delete p1;
79     A::showInfo( );
80     int n = 80;
81     A * p2 = new A[ n ];
82     A::showInfo( );
83     delete[ ] p2;
84     A::showInfo( );
85 }

```

运行结果:

```

累计创建的对象个数 = 1
累计释放的对象个数 = 0
当前存活的对象个数 = 1
存活对象所占内存数 = 4
所占用内存的最大值 = 4
累计创建的对象个数 = 1
累计释放的对象个数 = 1
当前存活的对象个数 = 0
存活对象所占内存数 = 0
所占用内存的最大值 = 4

```



```

累计创建的对象个数 = 81
累计释放的对象个数 = 1
当前存活的对象个数 = 80
存活对象所占内存数 = 320
所占用内存的最大值 = 320
累计创建的对象个数 = 81
累计释放的对象个数 = 81
当前存活的对象个数 = 0
存活对象所占内存数 = 0
所占用内存的最大值 = 320

```

11.6 使用中的注意事项

使用动态内存给程序员提供了更灵活的管理对象和数据的方法。在编程规范中，通常也禁止或限制在静态存储区存储大对象和大数组，特别地，在很多现代编程语言中，直接禁止在静态区创建对象，而只能在堆区创建对象。在 C++ 中使用动态内存时，还要注意以下问题：

- (1) 当用 `T * pT = new T;` 创建 T 类型对象后，一般使用 `delete pT;` 释放对象。但如果将 `pt` 赋给其它类型，再释放，可能出现问题。例如 `void * p = pT; delete p;`，这种情况下，类 T 的析构函数不会执行。
- (2) 当释放动态内存时，只执行释放所需的操作，但不会修改指针本身的值，这时的指针就成为悬浮指针，也称无效指针或野指针。如 `T * p = new T; delete p;` 之后，`p` 指向的内存空间已释放，但 `p` 指针本身的值没有修改，这时再进行 `p->f();` 的访问，会产生运行时的访问非法地址错误。因此，在执行完 `delete` 后，应确保不再访问指针，或明确给指针赋值为空。

例如：

```

1  struct B {
2      int mVal;
3  };
4  class A {
5  public:
6      static A * getInstance( ) {
7          if ( spA == nullptr ) spA = new A;
8          return spA;
9      }
10     static void release( ) {
11         delete spA;
12         //应增下面的语句，确保调用 release 后，仍可通过 getInstance 获取对象
13         spA = nullptr;

```

```

14     }
15 private:
16     A( ) {
17         mpB = new B;
18         mpB->mVal = 2;
19     }
20 public:
21     ~A( ) {
22         delete mpB;
23         //此函数结束前不应再访问 mpB
24     }
25 private:
26     static A * spA;
27     B * mpB;
28 };
29 A * A::spA = nullptr;
30 int main( ) {
31     A * pA1 = A::getInstance( );
32     A::release( );
33     A * pA2 = A::getInstance( );
34     A::release( );
35 }

```

(3) 使用动态内存时，最常见的也是最大的问题就是内存泄露。例如，执行 $T * p = new T$ ；之后，直到程序结束都没有执行 `delete p`；这就会产生内存泄漏。对大型项目来说，针对内存泄漏的测试是必不可少的，但现实是几乎都存在内存泄漏，而且很难完全避免。为此，应遵循：

- `new` 和 `delete` 应成对出现。使用 `new` 创建的对象，要由对应的 `delete` 释放；由 `new[]` 创建的，用 `delete[]` 释放。
- `new` 和 `delete` 在代码中的位置，应尽可能地近一些。例子代码中的 `A * pA1 = A::getInstance( )`；语句和 `A::release( )`；是相邻出现的，但在实际项目中它们中间可能会有其它更多的语句，甚至这两条语句都不在一个函数中。因此要严格遵守此条规范，有时是很难的。

从以上说明可以看出，使用动态内存是非常好设计思想，它给程序员最大的灵活性。但要正确使用动态内存，必须解决内存泄漏的问题，这不能仅仅依靠程序员的自身觉悟或通过要求程序员遵守编程规范来保证。为解决内存泄漏，常用以下几种方式：

1. 垃圾自动回收。由语言本身提供垃圾内存的自动回收机制，这也是很多现代语言采用的。

但由于回收的时机是不确定的,这种方式会产生类似"抽搐"的现象,这限制了语言的应用范围。

2. 指定对象所有者。创建每个对象时,都指定对象的所有者,当释放所有者对象时,自动释放其拥有的所有对象。这种方式要求对象间的层次关系必须明确,但在实际项目中,这种明确的关系是难以保证的,因此不具有通用性。该方式主要用在图形界面中各组件对象的设计上,如 Qt 的图形库组件。
3. 使用 `auto_ptr` 指针,也称智能指针或灵巧指针,应用于 C++ 早期标准中,由于 `auto_ptr` 本身存在一些问题,如安全性,现已在新标准中废弃,但理解其设计思想仍是必要的。这里的智能是指自动释放动态内存,其核心就是利用静态内存总会自动释放的特点,确保在堆区分配的内存总有释放的机会,例如:

```

1  class T {
2  public:
3      void f( ) {}
4  };
5  class Auto_ptr {
6  public:
7      Auto_ptr( T * pT ) : mpT( pT ) { }
8      ~Auto_ptr( ) { delete mpT; }
9      T * operator->( ) { return mpT; }
10     const T * operator->( ) const { return mpT; }
11     T & operator*( ) { return *mpT; }
12     const T & operator*( ) const { return *mpT; }
13     //其它略
14 private:
15     T * mpT;
16 };
17 int main( ) {
18     Auto_ptr ptr( new T );
19     ptr->f( );
20     (*ptr).f( );
21 }
```

无论 T 类型多么复杂, `auto_ptr` 类型的对象总是一个小对象,它只含有一个指针数据成员。

在 `main` 函数中,使用了 `new T` 表达式,虽然没有出现对应的 `delete` 表达式,但由于 `ptr` 是栈区对象,当函数返回时,会自动释放 `ptr` 对象,即一定会执行 `auto_ptr` 的析构函数,也就执行释放动态内存的操作。从 `auto_ptr` 的使用者看来,智能指针做到了"只管分配,不管释放"的智能性。

4. 使用新标准中的智能指针，包括共享指针、弱指针和唯一指针。详见下节。
5. 更通用的方法是采用 RAII(Resource Acquisition Is Initialization)技术，也称资源获取即初始化。实际上，auto_ptr 和智能指针等都是 RAII 技术的应用。详见本章后续资源获取即初始化一节。

11.7 引用计数和共享指针

11.7.1 共享及引用计数

假设在堆区有一个 A 类对象，而 User 类作为该对象的使用者，代码如下：

```

1  class A {
2  public:
3      A( int n = 0 ) : mData(n) { }
4      void f( );
5  private:
6      int mData;
7  };
8  class User {
9  public:
10     User( ) :mpA(new A) { }
11     ~User( ) { delete mpA; }
12     void g( ) { }
13  public:
14     A * operator->( )          { return mpA; }
15     const A * operator->( ) const { return mpA; }
16  private:
17     A * mpA;
18  };
19  int main( ) {
20     User user1;
21     user1.g( );
22     user1->f( );
23 }
```

上述代码虽然能够正确执行的，但 User 类是有缺陷的：User 类没有自定义的拷贝构造函数和赋值函数，所以使用编译器提供的缺省拷贝构造和缺省赋值函数，因此拷贝和赋值时，会采用“浅”复制和“浅”赋值，这导致释放 User 对象时，会产生重复释放的问题。要解决此缺陷，常见的方法就是自定义 User 类的拷贝和赋值函数，如：

```

1  class User {
```

```

2  public:
3      User( ) : mpA( new A ) { }
4      ~User( ) { delete mpA; }
5      void g( ) { }
6  public:
7      User(const User & rhs) { mpA = new A( *rhs.mpA); }
8      User & operator=( User rhs ) {
9          swap(mpA,rhs.mpA);
10         return *this;
11     }
12 public:
13     A * operator->( )    { return  mpA; }
14     const A * operator->( ) const { return  mpA; }
15 private:
16     A * mpA;
17 };

```

这又引出了新的疑问：首先，User 类真的有必要实现物理意义上的深拷贝和深赋值吗？如果类 A 是一个复杂的类，拷贝 A 类对象的资源及性能损耗能承受吗？其次，类 A 能拷贝吗？即使能够自定义类 A 的拷贝构造函数，真的能自定义实现吗？

为此，可引入共享的思想：当拷贝构造 User 对象时，让新 User 和旧 User 共享同一个 mpA，也就是“浅”拷贝的功能，同时增加一个计数，用于记录 mpA 被共享的次数。每共享一次，计数加 1，每释放一次，计数减 1。只有当计数为 0 时，才真正释放 mpA。这种思想的核心是在需要拷贝 A 类对象时，只进行逻辑意义上的“深”拷贝，而不是常规的物理意义上的“深”拷贝，实现上就是通过被记录的共享次数，来决定何时释放共享对象。这里的共享次数，也称为引用计数。

要实现共享，关键是管理引用计数。对此，可有两种实现方式。

方式一：在 A 类中增加一个实例变量，用于记录一个对象的被引用次数。首次实例化对象时，这个计数初始为 1；之后，每增加一次共享，计数加 1；当需要释放时，先进行计数的减 1 处理，只有计数减到 0 时，才真正释放对象。这样，对于三个 User 的处理过程，如图 11-1 所示：

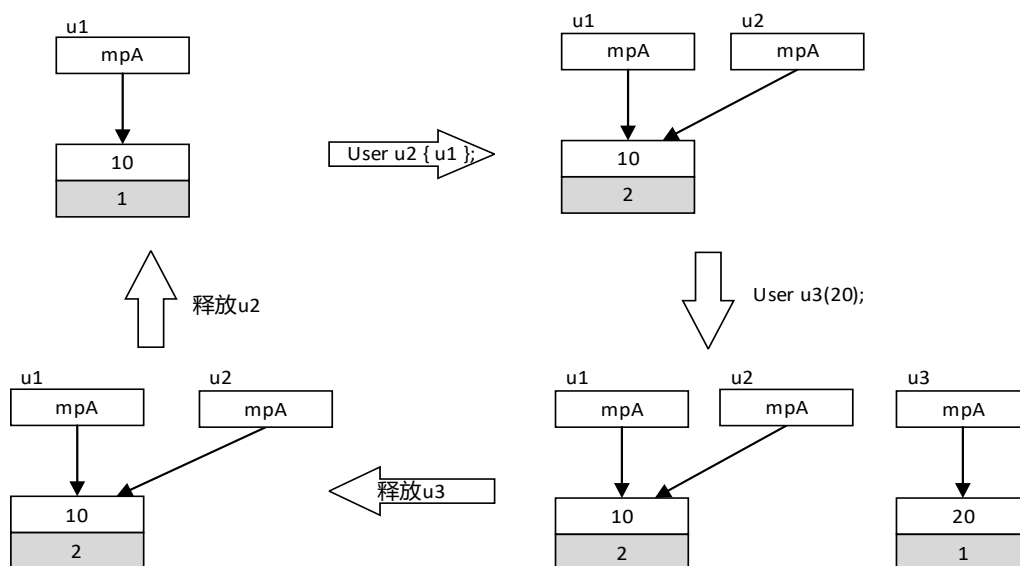


图 11-1 共享指针实现方式一示意图

具体实现代码如下：

```

1  #include <iostream>
2  class A {
3  public:
4      A(int n):mData( n ), mRefCount( 1 ) {}
5      void f( ) { }
6  public:
7      void addRef( ) { ++mRefCount; }
8      int  releaseRef( ) { return --mRefCount; }
9  private:
10     int  mData;
11     int  mRefCount = 1; //初始引用计数
12 };
13 class User {
14 public:
15     User( int n ) :mpA( new A( n ) ) { }
16     ~User( ) { release( ); }
17     void g( ) { }
18 public:
19     User( const User & rhs ) {
20         //mpA = new A( *rhs.mpA); 原实现
21         mpA = rhs.mpA;
22         mpA->addRef( );
23     }
24     User & operator=( User rhs ) {
25         std::swap( mpA, rhs.mpA );
26         return *this;

```

```
27     }
28 public:
29     A * operator->( ) { return mpA; }
30     const A * operator->( ) const { return mpA; }
31 private:
32     void release( ) {
33         if ( mpA->releaseRef( ) == 0 ) delete mpA;
34     }
35 private:
36     A * mpA;
37 };
38 int main( ) {
39     User u1( 10 );
40     User u2{ u1 };
41     User u3( 20 );
42 }
```

这种方式实现简单，但需要修改类 A 的源代码，即需要在类 A 中增加一个记录共享次数的计数变量，注意这个变量是实例变量，不能是类变量。若不能修改类 A，如没有源代码或者由于版权限制原因等，可以使用第二种方式。

方式二：由类 A 的使用方记录共享次数(如 User 类)，就不需要修改类 A 源码了。实现中，让每个 User 对象除了拥有一个 A 类指针之外，还拥有一个指向记录 A 类对象共享次数的指针。初始时，共享计数为 1；共享时，增加共享计数；释放 User 对象时，减少共享计数；当共享计数减至 0 时，就可以安全地释放 A 类对象了。这样，对于三个 User 的处理过程，如图 11-2 所示：

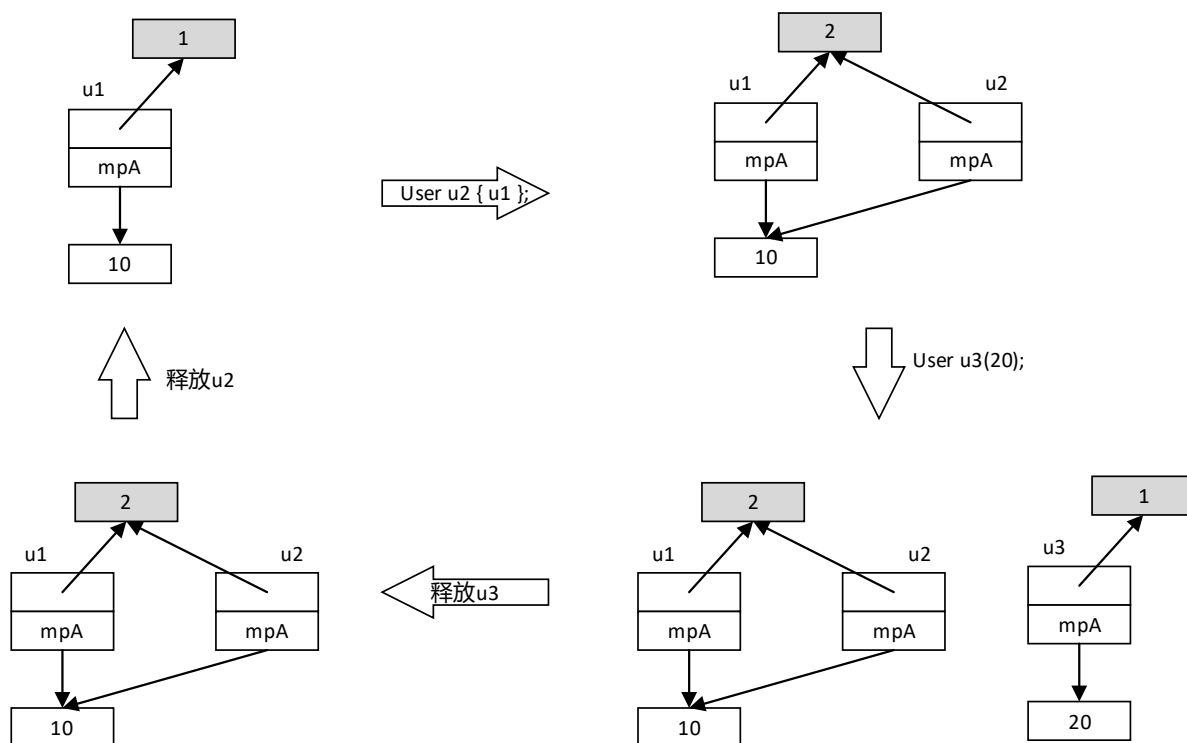


图 11-2 共享指针实现方式二示意图

具体实现代码如下:

```

1  #include <iostream>
2  class A {
3  public:
4      A( int n ) :mData( n ) { }
5      void f( ) { }
6  private:
7      int  mData;
8  };
9  class User {
10 public:
11     User( int n ) :mpA( new A( n ) ), mpRefCount( new int( 1 ) ) { }
12     ~User( ) { releaseRef( ); }
13     void g( ) { }
14 public:
15     User( const User & rhs ) {
16         mpA = rhs.mpA;
17         mpRefCount = rhs.mpRefCount;
18         addRef( );
19     }
20     User & operator=( User rhs ) {
21         std::swap( mpA, rhs.mpA );
22         std::swap( mpRefCount, rhs.mpRefCount );

```



```

23         return *this;
24     }
25 public:
26     A * operator->( ) { return mpA; }
27     const A * operator->( ) const { return mpA; }
28 private:
29     void addRef( ) {
30         ++( *mpRefCount );
31     }
32     void releaseRef( ) {
33         if ( --( *mpRefCount ) == 0 ) {
34             delete mpA;
35             delete mpRefCount;
36         }
37     }
38 private:
39     A * mpA;
40     int * mpRefCount = nullptr;
41 };
42 int main( ) {
43     User u1( 10 );
44     User u2{ u1 };
45     User u3( 20 );
46 }

```

相比于方式一，方式二的实现方式更优雅。首先，不用担心类 A 是否能拷贝和赋值，也不需要修改类 A 源码，当然也不改变类 A 的访问接口。其次，可以没有类 A 的源码，只要类 A 能实例化就可以，应用范围更广。

11.7.2 std::shared_ptr

在上述的第二种方式中，通过使用引用计数，可以共享动态内存中的对象。但在使用者 User 类中，还需要编写不少处理引用计数的代码，如 addRef()/releaseRef()函数，分配和释放 mpRefCount 等，使得定义 User 类变得更困难也更容易出错。可以考虑对与引用计数相关的数据和操作做进一步的抽象和封装，采用类似 auto_ptr 的方式，封装为如下的 SharedA 类：

```

1  #include <iostream>
2  class A {
3  public:
4      A( int n ) : mData( n ) { }
5      void f( ) { }
6  private:

```

```

7     int mData;
8 };
9 class SharedA {
10 public:
11     SharedA( A * pA ) : mpA(pA),mpRefCount(new int(1)){}
12     ~SharedA( ) { releaseRef( ); }
13     SharedA( const SharedA & rhs) {
14         mpA = rhs.mpA;
15         mpRefCount = rhs.mpRefCount;
16         addRef( );
17     }
18     SharedA & operator=( SharedA rhs ) {
19         std::swap( mpA, rhs.mpA );
20         std::swap( mpRefCount, rhs.mpRefCount );
21         return *this;
22     }
23 public:
24     A * operator->( ) { return mpA; }
25     const A * operator->( ) const { return mpA; }
26 private:
27     void addRef( ) { ++( *mpRefCount ); }
28     void releaseRef( ) {
29         if ( --( *mpRefCount ) == 0 ) {
30             delete mpA;
31             delete mpRefCount;
32         }
33     }
34 private:
35     A * mpA;
36     int * mpRefCount = nullptr;
37 };

```

可以看出，SharedA 类与原 User 类相比，只含有与共享 A 类指针相关的内容，不包含其它内容，如 User 中的 g() 不出现在 SharedA 类中。现在，基于 SharedA 类，可以如下重新定义 User 类，而 main 函数不用改变：

```

1 class User {
2 public:
3     User( int n ) :mA( new A( n ) ) { }
4     void g( ) { }
5     SharedA & operator->( ) { return mA;}
6     const SharedA & operator->( ) const { return mA;}
7 private:

```

```

8     SharedA  mA;
9 };
10 int main( ) {
11     User u1( 10 );
12     User u2{ u1 };
13     User u3( 20 );
14
15     u1.g( );
16     u2->f( );
17     u3->f( );
18 }

```

基于 SharedA 类，可以简化 User 类的定义，无需显式声明拷贝构造函数和赋值函数。访问 User 类对象的方式保持不变，并且不会发生内存泄漏。由于 mA 是 User 类的对象成员，在 User 对象被销毁时，会自动调用 mA 的析构函数，从而减少引用计数，当引用计数降为零时，A 类对象将自动释放。

同理，如果希望共享一个堆区中的 B 类对象，只需参照 SharedA 类定义一个 SharedB 类。进一步对比 SharedA 类和 SharedB 类的定义可以发现，这两个类的区别仅在于类型名不同：一个是 A，另一个是 B。因此，可以通过类模板统一定义这样的 SharedX 类，这就是标准库中的 shared_ptr。以下是 shared_ptr 类模板的简要定义，更完整的、严格的定义和使用方法请参考 C++ 标准。

```

1  template <typename T>
2  class shared_ptr {
3  public:
4      shared_ptr( ) : mpT( nullptr), mpRefCount(nullptr) { }
5      ~shared_ptr( ) { releaseRef( ); }
6      shared_ptr( const shared_ptr & rhs) {
7          mpT = rhs.mpT;
8          mpRefCount = rhs.mpRefCount;
9          addRef( );
10     }
11     shared_ptr & operator=( shared_ptr rhs ) {
12         std::swap( mpT, rhs.mpT );
13         std::swap( mpRefCount, rhs.mpRefCount );
14         return *this;
15     }
16 public:
17     void reset(T * ptr = nullptr) {
18         releaseRef( );
19         mpT = ptr;
20         if(ptr) {

```

```

21         mpRefCount = new int(1);
22     }
23 }
24 public:
25     T * get( ) const { return mpT; }
26     int useCount( ) const { return (mpRefCount?(*mpRefCount):0); }
27     operator bool( ) const { return !(!mpT); }
28     T & operator*( ) const { return *mpT; }
29     T * operator->( ) const { return mpT; }
30 private:
31     void addRef( ) { if ( mpT ) ++( *mpRefCount ); }
32     void releaseRef( ) {
33         if ( mpT && --( *mpRefCount ) == 0 ) {
34             delete mpT;
35             delete mpRefCount;
36             mpT = nullptr;
37             mpRefCount = nullptr;
38         }
39     }
40 private:
41     T * mpT = nullptr;
42     int * mpRefCount = nullptr;
43 };

```

要使用标准库中的 `std::shared_ptr` 类以及相关类和函数，需要包含 `<memory>` 头文件。以下是一个使用 `std::shared_ptr` 的示例：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4  class Number {
5  public:
6      Number( ) = default;
7      Number( int n ) : mNum( n ) { }
8      ~Number( ) {
9          static int order = 0;
10         cout << ++order << " Number::~~Number( )" << endl;
11     }
12     int twice( ) { mNum += mNum; return mNum; }
13     bool isOdd( ) const { return mNum % 2; }
14     bool isEven( ) const { return !isOdd( ); }
15 public:
16     int num( ) const { return mNum; }
17     void setNum( int n ) { mNum = n; }

```

```

18 private:
19     int mNum = 1;
20 };
21 class Client {
22 public:
23     Client( int n1, int n2 )
24         :mNum1( std::make_shared<Number>( n1 ) ), mNum2( std::make_shared<Number>( n2 ) )
25     { }
26     int sum( ) const { return mNum1->num( ) + mNum2->num( ); }
27 private:
28     std::shared_ptr<Number> mNum1;
29     std::shared_ptr<Number> mNum2;
30 };
31 int main( ) {
32     using SPNumber = std::shared_ptr<Number>;
33     SPNumber a0;
34     if ( a0 ) {
35         cout << "a0 != nullptr" << endl;
36     }else{
37         cout << "a0 == nullptr" << endl;
38     }
39     SPNumber a1;
40     a1.reset( new Number( 77 ) );
41     cout << "num = " << a1->num( ) << endl;
42     cout << "num is an even = " << boolalpha << a1->isEven( ) << endl;
43     cout << "num is an odd = " << boolalpha << a1->isOdd( ) << endl << endl;
44     SPNumber a2 = std::make_shared<Number>( );
45     cout << "num = " << a2->num( ) << endl;
46     cout << "num is an even = " << boolalpha << a2->isEven( ) << endl;
47     cout << "num is an odd = " << boolalpha << a2->isOdd( ) << endl << endl;
48     SPNumber a3 = std::make_shared<Number>( 99 );
49     cout << "num = " << a3->num( ) << endl;
50     cout << "num is an even = " << boolalpha << a3->isEven( ) << endl;
51     cout << "num is an odd = " << boolalpha << a3->isOdd( ) << endl << endl;
52     SPNumber arr[ 3 ] {
53         std::make_shared<Number>( ),
54         std::make_shared<Number>( 5 ),
55         std::make_shared<Number>( 8 ),
56     };
57     for ( SPNumber item : arr )
58         cout << item->num( ) << endl;
59     cout << endl;
60     Client client1( 50, 70 );
61     cout << "Client1 Sum = " << client1.sum( ) << endl;

```

```

62     Client client2( 80, 90 );
63     cout << "Client2 Sum = " << client2.sum( ) << endl;
64     const Client client3( client1 );
65     cout << "Client3 Sum = " << client3.sum( ) << endl;
66     client2 = client1;
67     cout << "Client2 Sum = " << client2.sum( ) << endl;
68 }

```

运行结果:

```

a0 = nullptr
num = 77
num is an even = false
num is an odd  = true
num = 1
num is an even = false
num is an odd  = true
num = 99
num is an even = false
num is an odd  = true
1
5
8
Client1 Sum = 120
Client2 Sum = 170
Client3 Sum = 170
1 Number::~~Number( )
2 Number::~~Number( )
Client2 Sum = 120
3 Number::~~Number( )
4 Number::~~Number( )
5 Number::~~Number( )
6 Number::~~Number( )
7 Number::~~Number( )
8 Number::~~Number( )
9 Number::~~Number( )
10 Number::~~Number( )

```

这个例子只是演示了使用模板类 `shared_ptr` 的常用方法。其中 `make_shared` 用于强制在堆区创建对象，可以有效防止误传入一个指向静态存储区的指针，也是标准中推荐用法。这个例子中，应特别注意堆区中 `Number` 对象的释放次数，由于共享的存在，实际只创建了 10 个 `Number`，最后也正确释放了 10 个 `Number`。

11.7.3 std::shared_ptr 的应用

使用 `shared_ptr` 可以有效简化内存管理和资源释放。例如，考虑定义一个包含宿舍信息的学生类：

```

1  class Dorm {
2  public:
3      void func( ) { }
4  };
5  class Student {
6  public:
7      Student( ){ mpDorm = new Dorm; }
8      ~Student( ) { delete mpDorm; }
9  public:
10     //禁止了拷贝和赋值
11     Student( const Student &) = delete;
12     Student & operator=( const Student &) = delete;
13 public:
14     void takeRest( ) { mpDorm->func( ); }
15 private:
16     Dorm * mpDorm = nullptr;
17 };

```

在 `Student` 类中，必须自定义拷贝和赋值函数，而不能使用缺省版本，否则可能会导致内存释放问题。在示例中，直接禁用了拷贝和赋值操作，虽然这避免了内存问题，但也限制了 `Student` 类的使用场景，降低了代码复用性。为此，可以考虑使用 `shared_ptr` 来管理资源，从而在保证内存安全的同时，提升类的灵活性和复用性。

```

1  class Dorm {
2  public:
3      void func( ) { }
4  };
5  class Student {
6  public:
7      Student( ){ mpDorm = make_shared<Dorm>( ); }
8      void takeRest( ) { mpDorm->func( ); }
9  private:
10     shared_ptr<Dorm> mpDorm { nullptr };
11 };
12 int main( ) {
13     Student s1;
14     s1.takeRest( );
15     Student s2;

```

```

16     s2.takeRest( );
17     Student s3( s2 );
18     s3.takeRest( );
19     s2 = s3;
20     s2.takeRest( );
21 }

```

使用 `shared_ptr` 后,可以简化 `Student` 类的定义:不仅可以使缺省的析构、拷贝和赋值函数,还无需担心内存释放的问题。进一步地,若不希望多个 `Student` 实例修改同一个 `Dorm`,可以通过引入写时复制(Copy on Write)技术来改进 `takeRest` 函数的实现。

下面再举一个例子:一名教师可以指导多名学生的毕业论文,但每个学生只能有一位指导教师。教师类的定义是给定的,初始代码如下:

```

1  //Teacher 类的定义是确定的
2  class Teacher {
3  public:
4      Teacher( int id ) :mID( id ) {}
5      Teacher( const Teacher & ) = delete;
6      Teacher & operator = ( const Teacher & ) = delete;
7      /*其它略*/
8  private:
9      const int mID;
10 };
11 //需要定义的 Student 类
12 class Student {
13 public:
14     Student( Teacher & t ) : mTeacher( t ) { }
15     void study( ) { }
16 private:
17     Teacher mTeacher;
18 };
19 //示例性主程序
20 int main( ) {
21     Teacher * pt = new Teacher(1001);
22     Student * ps1 = new Student( *pt );
23     Student * ps2 = new Student( *pt );
24     ps1->study( );
25     ps2->study( );
26     delete ps2;
27     delete ps1;
28     delete pt;
29 }

```


上述代码无法通过编译，原因是 `Teacher` 类禁止了拷贝构造。在构建 `Student` 对象时，无法初始化其数据成员 `mTeacher`。可以将数据成员改为指针形式，例如：

```
1 class Student {
2 public:
3     Student( Teacher & t ) : mpTeacher(&t) { }
4     void study( ) { }
5 private:
6     Teacher * mpTeacher;
7 };
```

此时，出现了一个新的顾虑：在 `Student` 对象的生命周期内，`Teacher` 对象必须保持有效。也就是说，在 `Student` 对象活跃期间，`study()` 函数中不能释放 `mpTeacher` 所指向的对象，并且在 `main` 函数中，`delete pt;` 语句必须放在 `delete ps2;` 和 `delete ps1;` 语句之后。

为了满足这些隐性要求，可以使用 `shared_ptr` 来改进定义和实现，例如：

```
1 //需要定义的 Student 类
2 class Student {
3 public:
4     Student( shared_ptr<Teacher> t ) : mpTeacher( t ) { }
5     void study( ) { }
6 private:
7     shared_ptr<Teacher> mpTeacher;
8 };
9 //示例性主程序
10 int main( ) {
11     shared_ptr<Teacher> pt = make_shared<Teacher>( 1001 );
12     shared_ptr<Student> ps1 = make_shared<Student>( pt );
13     shared_ptr<Student> ps2 = make_shared<Student>( pt );
14     //pt.reset( ); //即使增加此语句也无问题
15     ps1->study( );
16     ps2->study( );
17 }
```

在新设计中，无需再担心 `Teacher` 会被过早释放的问题，且在程序结束时确保所有资源得到完全释放。

11.7.4 `std::shared_ptr` 的问题

`std::shared_ptr` 是 C++ 中非常强大的工具，它通过引用计数机制自动管理动态内存的释放，极大地简化了内存管理的复杂性。然而，`shared_ptr` 也存在一些局限性，尤其是在处理循环引用

(Circular Reference)时，引用计数机制无法正确释放内存，从而导致内存泄漏。

下面是一个典型的循环引用示例，展示了 `shared_ptr` 在处理循环链表时的不足：

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4  struct Node;
5  using SPNode = shared_ptr<Node>;
6  struct Node {
7      Node( int n ) :mNum( n ) { }
8      ~Node( ) {
9          static int count = 0;
10         cout << ++count << " ~Node( )" << endl;
11     }
12     int  mNum;
13     SPNode  mpNext;
14 };
15 int main( ) {
16     SPNode head  = make_shared<Node>( 1 );
17     SPNode middle = make_shared<Node>( 2 );
18     SPNode tail  = make_shared<Node>( 3 );
19     //至此，head/middle/tail 的引用计数都为 1
20
21     //形成循环引用
22     head->mpNext = middle;
23     middle->mpNext = tail;
24     tail->mpNext = head;
25     //至此，head/middle/tail 中共享数据的引用计数都为 2
26
27     SPNode next = head;  //此语句后，head 中共享数据的引用计数为 3
28     //下面是一个循环
29     //第一次循环后，head 中共享数据的引用计数为 2,middle 的计数为 3,tail 的计数为 2
30     //第二次循环后，head 中共享数据的引用计数为 2,middle 的计数为 2,tail 的计数为 3
31     //第三次循环后，head 中共享数据的引用计数为 3,middle 的计数为 2,tail 的计数为 2
32     do {
33         cout << "Num in Node = " << next->mNum << endl;
34         next = next->mpNext;
35     } while ( next != head );
36     //tail->mpNext.reset( ); //缺少这句，则内存泄漏
37
38     //若无 tail->mpNext.reset( );，而是直接返回，那么会
39     //先释放栈区对象 next，此时的 next 与 head 共享整数 1，其计数为 3。释放后，计数变为 2
40     //再释放栈区对象 tail，此时共享整数 3，其计数为 2。释放后，计数变为 1

```

```

41 //再释放栈区对象 middle,此时共享整数 2, 其计数为 2。释放后, 计数变为 1
42 //最后释放栈区对象 head,此时共享整数 1, 其计数为 2。释放后, 计数变为 1
43 //此时, 堆区共有三个共享数据, 分别是整数 1, 整数 2 和整数 3, 它们的引用计数都为 1。
44 //由于引用计数不为 0, 不会执行释放操作, 造成内存泄漏
45 }

```

从代码分析中可知, 使用 `shared_ptr` 时, 一旦存在循环引用链, 链上各共享对象的引用计数永远不会减至 0, 会导致内存泄漏, 这也是使用 `shared_ptr` 必须注意的。要想避免此问题, 必须打破这种循环链, 至今还没有非常完美的解决方案。在 C++ 标准中, 提供了三种智能指针, 除了 `shared_ptr`, 还有 `weak_ptr` 和 `unique_ptr`, 由程序员组合使用这几种智能指针来打破循环链, 从而避免内存泄漏。

其中, `unique_ptr` 表示是拥有对象的唯一拥所有者; `shared_ptr` 表示多个对象可共同拥有一个共享对象, 这时的引用计数就代表所有者的个数; 相对于 `shared_ptr`, `weak_ptr` 表示一种非拥有的语义, 通过 `weak_ptr` 也可以访问共享对象, 但不具有共享对象的所有权, 也就是说 `weak_ptr` 的拷贝和赋值, 不会增加共享对象的引用计数。例如在上边循环链表的列子中, 可用 `shared_ptr` 实例化三个结点, 而用 `weak_ptr` 定义结点间的后继关系, 那么将正确释放共享对象, 如:

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4  struct Node;
5  using SPNode = shared_ptr<Node>;
6  struct Node {
7      Node( int n ) : mNum( n ) { }
8      ~Node( ) {
9          static int count = 0;
10         cout << ++count << " ~Node( ) " << endl;
11     }
12     int mNum;
13     weak_ptr<Node> mpNext; //使用了弱指针
14 };
15 int main( ) {
16     SPNode head = make_shared<Node>( 1 ); //使用共享指针实例化对象
17     SPNode middle = make_shared<Node>( 2 ); //使用共享指针实例化对象
18     SPNode tail = make_shared<Node>( 3 ); //使用共享指针实例化对象
19     head->mpNext = middle;
20     middle->mpNext = tail;
21     tail->mpNext = head;
22     weak_ptr<Node> next = head;
23     do {

```

```

24     SPNode temp = next.lock( );
25     cout << "Num in Node = " << temp->mNum << endl;
26     next = temp->mpNext;
27 } while ( next.lock( ) != head );
28 }

```

有关智能指针的进一步说明和使用方法，请参考 C++11 标准文档。

11.8 定位分配

C++还提供了一种特殊的 `new` 表达式用法，允许为 `new` 表达式传递额外参数。其语法格式如下：

格式 1: `[::] new (占位参数列表) T [初始化]`

格式 2: `[::] new (占位参数列表) (T) [初始化]`

当 `T` 为非数组类型时，表示创建单个对象；若 `T` 为数组类型，则表示创建数组。在创建和销毁对象时，会调用相应的 `operator new`、`operator delete`、`operator new[]` 和 `operator delete[]` 函数。由于引入了额外的参数列表，这些操作符函数需要自定义实现，无法使用系统默认版本。它们的定义格式如下：

格式 1: `void * operator new (std::size_t size, 占位参数表)`

格式 2: `void operator delete (void * ptr, 占位参数表) noexcept`

格式 3: `void * operator new[] (std::size_t size, 占位参数表)`

格式 4: `void operator delete[] (void * ptr, 占位参数表) noexcept`

通过这种方式，用户可以通过传递额外参数来满足特定需求。需要注意的是，大多数编译器支持带参数的 `operator new` 和 `operator new[]`，但对于 `operator delete` 和 `operator delete[]` 的带参数版本，部分编译器可能不完全支持，可能会退回到不带参数的标准版本。例如：

```

1  #include <iostream>
2  using namespace std;
3  class T {
4  public:
5      static void * operator new(size_t size) {           //对象分配
6          cout << "operator new(size_t size)" << endl;
7          return new char[ size ];
8      }
9      static void operator delete(void * ptr) noexcept { //对象释放
10         cout << "operator delete(void * ptr)" << endl;
11         delete[] reinterpret_cast<char *>(ptr);
12     }

```

```

13     static void * operator new(size_t size, int a, size_t & b) { //对象占位分配
14         cout << "operator new(size_t size, int a, size_t & b)" << endl;
15         b += size + sizeof( int ) * a;
16         return new char[ size + sizeof( int ) ];
17     }
18     static void operator delete(void * ptr, int a, size_t & b) noexcept { //对象占位释放
19         cout << "operator delete(void * ptr, int a, int b)" << endl;
20         delete[ ] reinterpret_cast<char *>(ptr);
21     }
22     static void * operator new[]( size_t size ) { //数组分配
23         cout << "operator new[](size_t size)" << endl;
24         return new char[ size ];
25     }
26     static void operator delete[]( void * ptr ) noexcept { //数组释放
27         cout << "operator delete[](void * ptr)" << endl;
28         delete[ ] reinterpret_cast<char *>(ptr);
29     }
30     static void * operator new[]( size_t size, int a, size_t & b ) { //数组占位分配
31         cout << "operator new[](size_t size,int a, size_t & b )" << endl;
32         b += size + sizeof( int ) * a;
33         return new char[ size + sizeof( a ) ];
34     }
35     static void operator delete[]( void * ptr, int a, size_t & b ) noexcept { //数组占位
36         cout << "operator delete[](void * ptr, int a, size_t & b)" << endl;
37         delete[ ] reinterpret_cast<char *>(ptr);
38     }
39
40     int mA;
41     int mB;
42 };
43
44 int main( ) {
45     size_t bytes = 0;
46     cout << "sizeof(T) = " << sizeof( T ) << endl; //输出 sizeof(T) = 8
47     T * pT1 = new T { 1,2 }; //输出 operator new(size_t size)
48     delete pT1; //输出 operator delete(void * ptr)
49
50     T * pT2 = new (2, bytes)T { 3,4 }; //输出 operator new(size_t size,int a,size_t& b)
51     delete pT2; //输出 operator delete(void * ptr)
52     cout << "bytes = " << bytes << endl; //输出 bytes = 16
53
54     T * pT3 = new (3,bytes)T{ 5,6 }; //输出 operator new(size_t size, int a, size_t & b)
55     delete pT3; //输出 operator delete(void * ptr)

```

```

56     cout << "bytes = " << bytes << endl;    //输出 bytes = 36
57     //输出 operator delete[](void * ptr, int a, size_t & b)
58     T * pT4 = new (4, bytes)T[ 5 ] { {1,2},{3,4},{5,6} };
59     delete[ ] pT4;                          //输出 operator delete[](void * ptr)
60     cout << "bytes = " << bytes << endl;    //输出 bytes = 92
61 }

```

例子中，通过占位参数可以获得累计分配的字节数。在释放内存时，理论上应调用与 `new` 匹配的 `operator delete` 或 `operator delete[]`，本例中应调用带占位参数的 `operator delete` 版本。然而，实际调用的是普通版本，这可能是由于编译器尚未完全支持 C++ 标准所致。由于这种情况仅涉及 `delete` 表达式的内部处理，对程序设计的实际影响较小。

通常情况下，如果没有特殊需求，很少使用带占位参数的 `new` 表达式。然而，在定位分配这种特殊场景中，确实需要使用这种形式。**定位分配**是 `new` 表达式的一种特殊使用形式。普通的 `new` 表达式会在堆区分配内存并构建对象，而定位 `new` 的特别之处在于：它不会分配新的内存，而是在已有的内存位置上直接构造对象或对象数组。这个已有的内存可以是动态分配的，也可以是静态分配的。使用定位 `new` 时，编译器会提供缺省的带定位参数的 `operator new` 和 `operator new[]`。定位分配的语法格式为：

格式： `new ( 定位参数 ) 类型 ( [ 初始表达式 ] )`

例如：

```

1  {
2      char * buf = new char[sizeof(T)]; //分配堆区内存
3      T * pT1 = new (buf) T(3,4);      //在指定的堆区位置上构建 T 类对象
4      pT1->~T( );                      //需要显式调用析构函数
5      delete[ ] buf;                  //释放堆区内存
6
7      char arr[sizeof(T)];             //在栈区分配内存
8      T * pT2 = new (arr) T(3,4);      //在栈区位置构建 T 类对象
9      pT2->~T( );                      //需要显式调用析构函数
10 }

```

下面是一个使用定位分配的例子：在构建对象栈数据结构时，可以一次性分配所需的全部内存空间。每次执行 `push` 操作时，在已分配的内存中构造新对象；而执行 `pop` 操作时，只需正确析构对象，无需释放内存空间。

```

1  #include <iostream>
2  using namespace std;
3  struct Item {

```

```

4     Item( int a = 0, int b = 0 ) :mA( a ), mB( b ) { }
5     ~Item( ) { cout << "Item::~~Item( )" << endl; }
6     void show( ) const {
7         cout << "Item's data are: ";
8         cout << "mA = " << mA << " , mB = " << mB << endl;
9     }
10    int mA;
11    int mB;
12 };
13 class Stack {
14 public:
15     Stack( int maxCount ) : mCapacity( maxCount ), mTop( 0 ) {
16         //一次性分配足够的空间
17         mpItems = ( Item * ) new char[ mCapacity * sizeof( Item ) ];
18     }
19     ~Stack( ) {
20         //析构时, 先执行各对象的析构函数,
21         while ( !empty( ) ) pop( );
22         //最后一次性释放存储空间
23         delete[ ]( char * )mpItems;
24     }
25     //禁止拷贝和赋值
26     Stack( const Stack & ) = delete;
27     Stack & operator=( const Stack & ) = delete;
28 public:
29     bool empty( ) const {
30         return ( mTop == 0 );
31     }
32     size_t count( ) const { return mTop; }
33     Item & top( ) {
34         if ( mTop == 0 ) throw "栈为空";
35         return mpItems[ mTop - 1 ];
36     }
37     void push( const Item & item ) {
38         if ( mTop + 1 >= mCapacity ) throw "栈溢出";
39         //在当前栈顶位置, 只构建新对象, 不需分配空间
40         Item * p = new ( &mpItems[ mTop++ ] ) Item( item );
41     }
42     void pop( ) {
43         if ( mTop == 0 ) return;
44         //析构栈顶对象, 但不能释放空间
45         ( &mpItems[ --mTop ] )->~Item( ); //显式调用析构,
46     }
47 private:

```

```

48     Item * mpItems = nullptr;
49     int      mTop = 0;          //栈顶位置
50     const int  mCapacity;      //最大容量
51 };
52 int main( ) {
53     Stack stack( 10 );
54     stack.push( Item( 1, 2 ) );
55     stack.push( Item( 3, 4 ) );
56     stack.push( Item( 5, 6 ) );
57     stack.push( Item( 7, 8 ) );
58     stack.top( ).show( );
59     stack.pop( );
60     stack.top( ).show( );
61 }

```

11.9 写时复制

写时复制(Copy on Write , COW)是程序设计中常用的一种编程技术。在程序中会出现一个系统资源被多个竞争者申请的情况,无论在单线程或多线程的设计中都会发生。这里的系统资源可以是同一块内存,也可以是文件,数据表等,通常系统资源是宝贵的,不会让每个竞争者都独立地拥有系统资源或资源副本,最常用的方法就是共享。那么,当一个竞争者修改了系统资源,其它竞争者将得到变更后的资源,这可能不是程序员需要的。写时复制就是在不需要修改系统资源时,直接访问共享的系统资源;而当需要修改资源时,先复制出资源的一个副本,并独立拥有该副本,而不再共享原资源,最后再对新的副本进行修改。

下面给出一个针对共享对象,进行写时复制的例子:

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4  struct A {
5      A( int n = 1 ) : mNum( n ) { }
6      ~A( ) { cout << "A::~A( )" << endl; }
7      int mNum;
8  };
9  class Client {
10 public:
11     Client( int n ) : mspA( std::make_shared<A>( n ) ) { }
12     //读操作,直接访问共享对象
13     int readA( ) const {
14         return mspA->mNum;

```



```

15     }
16     //写操作,应用 copy on write
17     void modifyA( int added ) {
18         if ( mspA.use_count( ) > 1 ) {
19             //取得共享对象,经拷贝构造后新建一个副本,
20             //以此副本构建新的匿名的 shared_ptr 对象,新对象的引用计数为 1
21             //再将新对象赋值给 mspA。
22             //赋值时,会减少原 mspA 的引用计数,同时增加新对象的计数为 2,并赋值给 mspA
23             //赋值语句结束时,匿名的新 shared_ptr 对象释放,mspA 的引用计数减为 1
24             mspA = make_shared<A>( *mspA.get( ) );
25         }
26         mspA->mNum += added;
27     }
28 private:
29     std::shared_ptr<A> mspA;
30 };
31 int main( ) {
32     Client c1( 80 );
33     Client c2( c1 );
34     cout << c1.readA( ) << endl;
35     cout << c2.readA( ) << endl;
36     c2.modifyA( 100 );
37     cout << c1.readA( ) << endl;
38     cout << c2.readA( ) << endl;
39 }

```

11.10 资源获取即初始化(RAII)

在程序运行过程中,会动态申请并使用多种系统资源,如堆内存、执行线程、打开的套接字、文件句柄、互斥锁、磁盘空间以及数据库连接等。这些资源是有限的,使用完毕后必须及时释放并归还给系统;否则,可能导致内存泄漏、死锁、资源耗尽或程序异常等问题。

为了确保资源能够被正确回收,传统做法依赖编码规范进行约束。例如,规定使用 `new` 分配堆内存后,必须在适当位置调用 `delete` 进行释放。然而,这种人为约定存在明显缺陷:一方面,完全遵守规范需要大量严格的测试验证,开发成本高;另一方面,即使代码看似符合规范,在异常发生时仍可能无法保证资源释放——比如在 `new` 之后、`delete` 之前抛出异常,导致 `delete` 永远不会被执行,从而引发资源泄漏。

为解决这一问题,C++ 引入了 RAII (Resource Acquisition Is Initialization) 技术,即“资源获取即初始化”。RAII 的核心思想是:将资源的生命周期与某个对象的生命周期绑定,通过对象的构造和析构机制自动管理资源的获取与释放,从而确保无论函数正常退出还是因异常提前返回,资源

都能被安全、及时地释放。RAII 的实现包含两个关键点：

1. **资源封装**：将资源的申请和释放封装在一个类中。在构造函数中获取资源，在析构函数中释放对应的资源。由于 C++ 保证局部对象在离开作用域时自动调用析构函数，因此只要对象生命周期结束，资源就会被自动清理。
2. **使用具有自动生命周期的对象**：通过栈上对象或临时对象来使用 RAII 类。这类对象的生命周期由作用域控制，无需手动干预，即使发生异常，C++ 的栈展开机制也会确保其析构函数被调用，从而实现异常安全的资源管理。

例如，文件是一种资源，打开文件后应确保关闭文件。下面 File 类是应用 RAII 的一个示例：

```

1  class File {
2  public:
3      File( const string& fname ) { mHandler = openFile( fname ); }
4      ~File()      { closeFile( mHandler ); }
5      /* File 操作略 */
6  private:
7      Handler  mHandler;
8  };
9  int main( ) {
10     File file("C:\\my.txt");
11     /* 操作 file 对象略 */
12 }
```

实例化 file 对象时，用文件句柄记录了分配的文件资源；当 main 函数结束时，自动释放 file 对象，一定会调用 File 类的析构函数，从而确保了文件资源的回收。

再比如，std::shared_ptr 也是使用 RAII 技术的一个例子。

```

1  #include <memory>
2  class T {
3  public:
4      void doAnything( );
5      /* 其它略 */
6  };
7  int main( ) {
8      std::shared_ptr< T > spInt = std::make_shared<T>( );
9      spInt->doAnything( );
10 }
```

首先，在堆区创建一个 T 类型的对象，并使用该对象的地址来初始化栈上的共享指针对象 spInt。在此过程中，如果 T 类对象的构造失败并抛出异常，由于构造函数尚未完成，动态分配的内存将由

语言运行时机制或共享指针自身的异常安全保证自动回收，从而避免内存泄漏；若 T 类对象成功构造，则 `spInt` 被正确初始化。随后通过 `spInt` 调用 `doAnything()` 函数，无论该函数是否抛出异常，当控制流返回至 `main` 函数并最终退出时，都会触发栈的自动清理过程。此时，栈上对象 `spInt` 会随着作用域的结束而被销毁，其析构函数被自动调用，进而将引用计数减为零，并释放其所管理的堆上 T 类对象的内存。由此可见，`spInt` 的生命周期与堆内存的生命周期完全同步：只要 `spInt` 存在，堆内存就处于有效分配状态；一旦 `spInt` 被销毁，堆内存也将被安全释放，从而确保了内存管理的安全性和异常安全性。

综上，RAII 不仅有效防止了资源泄漏，还使代码更加简洁、健壮，并天然支持异常安全，是现代 C++ 资源管理的基本方法。

11.11 动态容器例

现定义一个存放多组交易数据的容器类。当添加新的交易数到容器时，如果容器容量不足，可自动扩充容器。标准库中的 `vector` 实际上就是这样的容器。

分析：容器类应含有容量、交易数据的项数以及具体数据；由于项数的不定，不适合采用静态数组，本例中没有直接使用标准库中的容器，而使用在堆区动态分配的内存记录数据；有了指针数据成员，考虑自定义实现拷贝和赋值；添加数据项时，为方便链式访问，可返回引用；通过重载[] 运算符实现取分量操作，且区分带 `const` 修饰和不带 `const` 修饰。代码如下：

```

1  // 接口文件
2  #pragma once
3  #include <iostream>
4  struct Item {
5      double open;      //开盘价
6      double close;     //收盘价
7      double high;      //最高价
8      double low;       //最低价
9      double volume;    //成交量
10 };
11 //输出 Item
12 std::ostream & operator<<( std::ostream & out, const Item & item );
13
14 class Items {
15 public:
16     Items( );    //构造
17     ~Items( );   //析构
18     Items( const Items & rhs );    //拷贝

```

```

19     Items & operator=( Items rhs );          //赋值
20 public:
21     Items & add( const Item & item );        //添加
22     Item & operator[ ]( size_t index );      //取分量
23     const Item & operator[ ]( size_t index )const;//取分量
24     size_t size( ) const { return mSize; } //项目数
25 private:
26     int      mCapacity; //最大容量
27     size_t   mSize;     //大小
28     Item *   mItems;    //价格集
29 };

```

```

1  // 实现文件
2  #include <iostream>
3  #include <iomanip>
4  #include <cassert>
5  using namespace std;
6  #include "Items.h"
7
8  //流输出
9  std::ostream & operator<<( std::ostream & out, const Item & item )
10 {
11     out << fixed << setprecision( 2 ) << showpoint;
12     out << "Open   :" << setw( 10 ) << item.open << endl;
13     out << "Close  :" << setw( 10 ) << item.close << endl;
14     out << "High   :" << setw( 10 ) << item.high << endl;
15     out << "Low    :" << setw( 10 ) << item.low << endl;
16     out << "Volume :" << setw( 10 ) << item.volume << endl << endl;
17     return out;
18 }
19 //构造函数，为演示效果，将初始容量设为 2
20 Items::Items( )
21     :mCapacity( 2 ), mSize( 0 )
22 {
23     mItems = new Item[ mCapacity ] { };
24 }
25 //析构函数
26 Items::~Items( )
27 {
28     delete[ ] mItems;
29 }
30 //拷贝构造函数
31 Items::Items( const Items & rhs )

```

```

32     :mCapacity( rhs.mCapacity ), mSize( rhs.mSize )
33 {
34     mItems = new Item[ mCapacity ] {};
35     for (size_t i = 0; i < mSize; ++i) {
36         *( mItems + i ) = rhs.mItems[ i ];
37         //mPrices[ i ] = rhs.mPrices[ i ]; //可能给出溢出警告
38     }
39 }
40 //赋值函数，注意参数是以传值方式传递的
41 Items & Items::operator=( Items rhs )
42 {
43     swap( mCapacity, rhs.mCapacity );
44     swap( mSize, rhs.mSize );
45     swap( mItems, rhs.mItems );
46     return *this;
47 }
48 //添加新项
49 Items & Items::add( const Item & item )
50 {
51     if (mSize >= mCapacity) {
52         Item * newItems = new Item[ mCapacity * 2 + 1 ] { }; //扩容到 2 倍+1
53         memcpy_s( newItems, sizeof( Item ) * mSize, mItems, sizeof( Item ) * mSize );
54         delete[ ] mItems;
55         mCapacity = mCapacity * 2 + 1;
56         mItems = newItems;
57     }
58     *( mItems + mSize ) = item;
59     ++mSize;
60     return *this;
61 }
62 //取分量
63 Item & Items::operator[]( size_t index )
64 {
65     assert( index < mSize );
66     return mItems[ index ];
67 }
68 //取分量
69 const Item & Items::operator[]( size_t index ) const
70 {
71     assert( index < mSize );
72     return mItems[ index ];
73 }

```

```

1 // 主函数
2 #include <iostream>
3 #include "Items.h"
4
5 int main( ) {
6     Items items;
7     items.add( { 12.40, 12.70, 12.85, 12.02, 340.50 } )
8         .add( { 12.50, 12.48, 12.80, 12.30, 280.31 } )
9         .add( { 12.40, 12.80, 12.99, 12.40, 350.82 } )
10        .add( { 12.62, 12.92, 13.11, 12.60, 390.00 } )
11        .add( { 12.80, 13.20, 13.48, 12.71, 370.78 } )
12        .add( { 13.40, 13.77, 13.95, 13.22, 380.63 } );//链式添加
13
14     std::cout << "Size =" << items.size( ) << std::endl;
15     for (int i = 0; i < items.size( ); ++i)
16         std::cout << items[ i ] << std::endl;
17 }

```

上述代码是一个自定义实现的容器类。在实际项目中,通常用标准库中的容器取代自定义容器,例如可使用 `std::vector<Item>` 取代自定义的 `Items` 类,从而简化代码实现。这时的代码可如下:

```

1 //用 vector 取代自定义的 Items
2 #include <iostream>
3 #include <iomanip>
4 #include <vector>
5 using namespace std;
6
7 struct Item {
8     double open;        //开盘价
9     double close;       //收盘价
10    double high;         //最高价
11    double low;          //最低价
12    double volume;       //成交量
13 };
14
15 std::ostream& operator<<( std::ostream& out, const Item& item ) {
16     out << fixed << setprecision( 2 ) << showpoint;
17     out << "Open   :" << setw( 10 ) << item.open << "\n";
18     out << "Close  :" << setw( 10 ) << item.close << "\n";
19     out << "High   :" << setw( 10 ) << item.high << "\n";
20     out << "Low    :" << setw( 10 ) << item.low << "\n";
21     out << "Volume :" << setw( 10 ) << item.volume << "\n\n";
22     return out;
23 }

```

```
24
25 int main( ) {
26     std::vector<Item> items;
27     items.emplace_back( Item { 12.40, 12.70, 12.85, 12.02, 340.50 } );
28     items.emplace_back( Item { 12.50, 12.48, 12.80, 12.30, 280.31 } );
29     items.emplace_back( Item { 12.40, 12.80, 12.99, 12.40, 350.82 } );
30     items.emplace_back( Item { 12.62, 12.92, 13.11, 12.60, 390.00 } );
31     items.emplace_back( Item { 12.80, 13.20, 13.48, 12.71, 370.78 } );
32     items.emplace_back( Item { 13.40, 13.77, 13.95, 13.22, 380.63 } );
33
34     cout << "Size = " << items.size( ) << endl;
35     for ( Item& item : items )
36         cout << item << endl;
37 }
```

使用标准容器可以简化程序设计工作。但对比自定义容器的实现，可以发现操作容器的数据项通常是通过传值实现的，因此，在使用 `vector` 容器时，一般需要 `Item` 类具有可访问的拷贝构造和赋值函数，这也是常用 `vector<T*>` 的形式存放多个数据项的原因。